

- DeepLearn: Technical Architecture & Addiction Mechanics
 - Making Learning as Addictive as TikTok
 - Table of Contents
- PART 1: VIDEO RECOMMENDATION ENGINE
 - The Challenge
 - The Solution: 3-Tier Recommendation System
 - Tier 1: YouTube API Smart Search (Primary)
 - Tier 2: AI-Curated Learning Paths
 - Tier 3: Collaborative Filtering (Advanced)
- PART 2: AI QUIZ GENERATION SYSTEM
 - The Complete Quiz Pipeline
 - Step 1: Extract Video Transcript
 - Step 2: Generate Quiz with Claude AI
 - Step 3: Adaptive Difficulty
 - Step 4: Quiz Delivery & Evaluation
- PART 3: ADDICTION & ENGAGEMENT MECHANICS
 - How TikTok Hooks Users (And How We Apply It)
 - TikTok's Addiction Playbook:
 - Our Ethical Adaptation:
 1. INSTANT GRATIFICATION
 2. VARIABLE REWARDS
 3. STREAKS (FOMO + Loss Aversion)
 4. SOCIAL PROOF & COMPETITION
 5. PROGRESS VISUALIZATION
 6. PUSH NOTIFICATIONS (Perfectly Timed)
 7. ONBOARDING THAT HOOKS
- PART 4: COMPLETE TECHNICAL STACK
 - Architecture Diagram
 - Database Schema
 - API Endpoints
 - Tech Stack Summary
- PART 5: 12-WEEK IMPLEMENTATION ROADMAP
 - Week 1-2: Foundation
 - Week 3-4: AI Quiz Generation
 - Week 5-6: Core Learning Loop
 - Week 7-8: Engagement Features
 - Week 9-10: Social & Monetization
 - Week 11-12: Polish & Launch
- CONCLUSION
 - What Makes DeepLearn Different
 - Success Metrics (6 Months)
 - The Vision

DeepLearn: Technical Architecture & Addiction Mechanics

Making Learning as Addictive as TikTok

Table of Contents

1. Video Recommendation Engine
 2. AI Quiz Generation System
 3. Addiction & Engagement Mechanics
 4. Complete Technical Stack
 5. Database Schema
 6. API Documentation
 7. Implementation Roadmap
-

PART 1: VIDEO RECOMMENDATION ENGINE

The Challenge

Users need the RIGHT videos, not just ANY videos. Bad recommendations = wasted time = churn.

The Solution: 3-Tier Recommendation System

Tier 1: YouTube API Smart Search (Primary)

How It Works:

```
async function recommendVideos(topic, userLevel = 'beginner') {
  // Step 1: Call YouTube Data API v3
  const searchResponse = await fetch(
    `https://www.googleapis.com/youtube/v3/search?` +
    `part=snippet&` +
    `q=${encodeURIComponent(topic)}&` +
    `type=video&` +
    `videoDuration=medium&` + // 4-20 minutes (sweet spot)
    `videoDefinition=high&` + // HD only
    `relevanceLanguage=en&` +
    `safeSearch=strict&` +
    `order=relevance&` + // Most relevant first
    `maxResults=50&` +
    `key=${YOUTUBE_API_KEY}`
  );

  const videos = await searchResponse.json();
  const videoIds = videos.items.map(v => v.id.videoId).join(',');

  // Step 2: Get detailed statistics
  const statsResponse = await fetch(
    `https://www.googleapis.com/youtube/v3/videos?` +
    `part=statistics,contentDetails,snippet&` +
    `id=${videoIds}&` +
    `key=${YOUTUBE_API_KEY}`
  );

  const detailedVideos = await statsResponse.json();

  // Step 3: Calculate Quality Score
  const scoredVideos = detailedVideos.items.map(video => {
    const stats = video.statistics;
    const duration = parseDuration(video.contentDetails.duration);

    // Quality Score Algorithm
    let score = 0;

    // 1. View Count (log scale to prevent mega-viral videos
    // dominating)
    const views = parseInt(stats.viewCount);
    score += Math.log10(views) * 10; // Max ~70 points for 10M+
    // views

    // 2. Like Ratio (critical quality indicator)
    const likes = parseInt(stats.likeCount);
    const dislikes = parseInt(stats.dislikeCount || 0);
    const likeRatio = likes / (likes + dislikes + 1);
    score += likeRatio * 50; // Max 50 points for 100% likes

    // 3. Engagement Rate
    const comments = parseInt(stats.commentCount);
    const engagement = (likes + comments) / views;
    score += Math.min(engagement * 10000, 20); // Max 20 points

    // 4. Optimal Duration (10-30 minutes is sweet spot)
    if (duration >= 10 && duration <= 30) {
      score += 25;
    } else if (duration >= 5 && duration < 10) {
      score += 15;
    } else if (duration > 30 && duration <= 60) {
      score += 10;
    }
  });
}
```

```

// 5. Recency Bonus (prefer content from last 2 years)
const ageInDays = (Date.now() - new
Date(video.snippet.publishedAt)) / (1000 * 60 * 60 * 24);
if (ageInDays < 365) {
  score += 15;
} else if (ageInDays < 730) {
  score += 10;
}

// 6. Educational Channel Bonus
const educationalChannels = [
  'Fireship', 'Traversy Media', 'freeCodeCamp',
  'Web Dev Simplified', 'The Net Ninja', 'Academind',
  'Programming with Mosh', 'Corey Schafer'
];
if (educationalChannels.includes(video.snippet.channelTitle)) {
  score += 30; // Big boost for known quality
}

return {
  videoId: video.id,
  title: video.snippet.title,
  channel: video.snippet.channelTitle,
  duration: duration,
  views: views,
  likeRatio: likeRatio,
  score: score,
  thumbnail: video.snippet.thumbnails.high.url
};
});

// Step 4: Sort by score and return top 10
return scoredVideos
  .sort((a, b) => b.score - a.score)
  .slice(0, 10);
}

function parseDuration(isoDuration) {
  // Convert ISO 8601 duration to minutes
  // PT15M30S -> 15.5 minutes
  const match = isoDuration.match(/PT(?:\d+H)?(?:\d+M)?(?:\d+S)?/);
  const hours = parseInt(match[1] || 0);
  const minutes = parseInt(match[2] || 0);
  const seconds = parseInt(match[3] || 0);
  return hours * 60 + minutes + seconds / 60;
}

```

Quality Score Breakdown:

Factor	Max Points	Why It Matters
View Count (log)	70	Popularity indicator
Like Ratio	50	Quality signal
Engagement Rate	20	Active learning community
Optimal Duration	25	Not too short, not too long
Recency	15	Up-to-date information
Educational Channel	30	Trusted creators
TOTAL	210	Higher = better

Tier 2: AI-Curated Learning Paths

For better recommendations, use Claude to create structured paths:

```

async function generateLearningPath(topic, userLevel, duration = 30)
{
  const prompt = `You are an expert curriculum designer.

  Create a ${duration}-day learning path for: ${topic}
  User level: ${userLevel}

```

For each day, provide:

1. Day number
2. Topic title (specific concept to learn)
3. 3 YouTube search queries (to find relevant videos)
4. Key learning objectives

Return as JSON:

```
{
  "path": [
    {
      "day": 1,
      "topic": "React Basics: Components and Props",
      "searchQueries": [
        "react components tutorial beginner",
        "react props explained",
        "functional components react"
      ],
      "objectives": [
        "Understand what components are",
        "Learn how to pass props",
        "Create your first component"
      ]
    }
  ]
}
```

```
const response = await
fetch('https://api.anthropic.com/v1/messages', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 4000,
    messages: [{ role: 'user', content: prompt }]
  })
});

const data = await response.json();
const learningPath = JSON.parse(data.content[0].text);

// For each day, search YouTube and pick best video
for (let day of learningPath.path) {
  const videos = [];

  for (let query of day.searchQueries) {
    const results = await recommendVideos(query, userLevel);
    videos.push(...results);
  }

  // Pick the highest-scored video
  day.video = videos.sort((a, b) => b.score - a.score)[0];
}

return learningPath;
}
```

Example Output:

```
{
  "path": [
    {
      "day": 1,
      "topic": "React Basics: What is React?",
      "video": {
        "videoId": "abc123",
        "title": "React in 100 Seconds",
        "channel": "Fireship",
        "duration": 2.5,
        "score": 195
      },
      "objectives": ["Understand React's purpose", "Learn JSX
basics"]
    },
    {
```

```

    "day": 2,
    "topic": "Components and Props",
    "video": {
      "videoId": "xyz789",
      "title": "React Components Tutorial",
      "channel": "Web Dev Simplified",
      "duration": 18,
      "score": 187
    }
  }
]
}

```

Tier 3: Collaborative Filtering (Advanced)

Learn from user behavior to improve recommendations:

```

// Track what videos users complete and retain
async function trackCompletion(userId, videoId, quizScore,
timeWatched) {
  await supabase.from('video_completions').insert({
    user_id: userId,
    video_id: videoId,
    quiz_score: quizScore,
    time_watched: timeWatched,
    completed: true,
    completed_at: new Date()
  });

  // Update video quality metrics
  await updateVideoMetrics(videoId, quizScore);
}

async function updateVideoMetrics(videoId, quizScore) {
  // Videos that lead to high quiz scores are QUALITY
  const { data: video } = await supabase
    .from('video_metrics')
    .select('*')
    .eq('video_id', videoId)
    .single();

  const newAvgScore = (
    (video.avg_quiz_score * video.completion_count) + quizScore
  ) / (video.completion_count + 1);

  await supabase
    .from('video_metrics')
    .update({
      avg_quiz_score: newAvgScore,
      completion_count: video.completion_count + 1,
      quality_score: calculateQualityScore(newAvgScore,
video.completion_count)
    })
    .eq('video_id', videoId);
}

function calculateQualityScore(avgQuizScore, completionCount) {
  // Videos with high quiz scores + many completions = GOLD
  return (avgQuizScore / 100) * Math.log10(completionCount + 1) *
50;
}

// Use this data to re-rank recommendations
async function getTopQualityVideos(topic) {
  const { data: videos } = await supabase
    .from('video_metrics')
    .select('*')
    .ilike('topic', `${topic}%`)
    .gte('completion_count', 10) // At least 10 completions
    .gte('avg_quiz_score', 75) // At least 75% avg score
    .order('quality_score', { ascending: false })
    .limit(10);
}

```

```
    return videos;
  }
```

The Feedback Loop:

User completes video
↓
Takes quiz (e.g., scores 85%)
↓
System learns: "This video leads to good comprehension"
↓
Video gets boosted in future recommendations
↓
More users see quality content
↓
Platform quality improves over time

PART 2: AI QUIZ GENERATION SYSTEM

The Complete Quiz Pipeline

Step 1: Extract Video Transcript

```
import { YoutubeTranscript } from 'youtube-transcript';

async function extractTranscript(videoId) {
  try {
    // Get transcript from YouTube
    const transcript = await
YoutubeTranscript.fetchTranscript(videoId);

    // Convert to plain text with timestamps
    const segments = transcript.map(item => ({
      text: item.text,
      timestamp: item.offset / 1000, // Convert to seconds
      duration: item.duration / 1000
    }));

    // Join into full text
    const fullText = segments.map(s => s.text).join(' ');

    // Clean up
    const cleanedText = fullText
      .replace(/\[Music\]/g, '')
      .replace(/\[Applause\]/g, '')
      .replace(/\s+/g, ' ')
      .trim();

    return {
      fullText: cleanedText,
      segments: segments,
      wordCount: cleanedText.split(' ').length
    };
  } catch (error) {
    // Fallback: No transcript available
    return null;
  }
}
```

Step 2: Generate Quiz with Claude AI

```
async function generateQuiz(videoId, transcript, difficulty =
'medium') {
  // Check cache first (save AI costs)
  const cachedQuiz = await supabase
    .from('quizzes')
    .select('*')
```

```

        .eq('video_id', videoId)
        .single();

    if (cachedQuiz.data) {
        return cachedQuiz.data.questions;
    }

    // Generate new quiz
    const prompt = `You are an expert educator creating comprehension
quizzes.

```

```

Video Transcript:
${transcript.fullText.substring(0, 8000)} // Limit to ~8K chars

Create 5 multiple-choice questions that test UNDERSTANDING, not
memorization.

```

```

Requirements:
- Difficulty: ${difficulty}
- 4 options per question (A, B, C, D)
- Only 1 correct answer
- Questions should require applying concepts, not reciting facts
- Include detailed explanations for correct answers
- Mix question types:
  * 2 conceptual (understanding "why")
  * 2 application (understanding "how")
  * 1 synthesis (connecting ideas)

```

```

Return ONLY valid JSON (no markdown, no backticks):
{
  "questions": [
    {
      "id": 1,
      "question": "Why does React use a virtual DOM instead of
directly manipulating the real DOM?",
      "options": {
        "A": "Because virtual DOM is faster to create",
        "B": "To batch updates and minimize expensive DOM
operations",
        "C": "To make the code easier to read",
        "D": "Because browsers don't support direct DOM
manipulation"
      },
      "correctAnswer": "B",
      "explanation": "React uses a virtual DOM to batch multiple
changes together and update the real DOM in one efficient operation,
avoiding the performance cost of many small DOM manipulations.",
      "difficulty": "medium",
      "type": "conceptual",
      "timestamp": 180
    }
  ]
};

```

```

const response = await
fetch('https://api.anthropic.com/v1/messages', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'anthropic-version': '2023-06-01',
    'x-api-key': process.env.ANTHROPIC_API_KEY
  },
  body: JSON.stringify({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 2500,
    temperature: 0.7, // Some creativity, but consistent
    messages: [{
      role: 'user',
      content: prompt
    }]
  })
});

const data = await response.json();

```

```

    // Parse response (handle potential markdown wrapping)
    let quizText = data.content[0].text.trim();
    quizText = quizText.replace(/```json\n?/g, '').replace(/```n?/g,
    '');

    const quiz = JSON.parse(quizText);

    // Cache the quiz for future users
    await supabase.from('quizzes').insert({
      video_id: videoId,
      questions: quiz.questions,
      difficulty: difficulty,
      generated_at: new Date()
    });

    return quiz.questions;
  }
}

```

Step 3: Adaptive Difficulty

Adjust quiz difficulty based on user performance:

```

async function getAdaptiveQuiz(userId, videoId, transcript) {
  // Check user's recent quiz performance
  const { data: recentQuizzes } = await supabase
    .from('quiz_attempts')
    .select('score')
    .eq('user_id', userId)
    .order('completed_at', { ascending: false })
    .limit(5);

  // Calculate average performance
  const avgScore = recentQuizzes.length > 0
    ? recentQuizzes.reduce((sum, q) => sum + q.score, 0) /
    recentQuizzes.length
    : 70; // Default to medium

  // Determine difficulty
  let difficulty;
  if (avgScore >= 90) {
    difficulty = 'hard';
  } else if (avgScore >= 70) {
    difficulty = 'medium';
  } else {
    difficulty = 'easy';
  }

  // Generate quiz at appropriate difficulty
  return await generateQuiz(videoId, transcript, difficulty);
}

```

Step 4: Quiz Delivery & Evaluation

```

// Frontend: Quiz Component
function QuizInterface({ videoId, questions }) {
  const [currentQ, setCurrentQ] = useState(0);
  const [answers, setAnswers] = useState({});
  const [showResults, setShowResults] = useState(false);
  const [score, setScore] = useState(null);

  const handleSubmit = () => {
    // Calculate score
    let correct = 0;
    questions.forEach((q, idx) => {
      if (answers[idx] === q.correctAnswer) {
        correct++;
      }
    });

    const percentage = (correct / questions.length) * 100;
    setScore(percentage);
  };
}

```



```

// Save to database
saveQuizAttempt(videoId, answers, percentage);

// Check if passed
if (percentage >= 80) {
  unlockNextVideo();
  showCelebration(); // Dopamine hit!
} else {
  showEncouragement(); // Supportive feedback
}

setShowResults(true);
};

return (
  <div className="quiz-container">
    {!showResults ? (
      <>
        <ProgressBar current={currentQ + 1} total=
{questions.length} />

        <QuestionCard
          question={questions[currentQ]}
          selectedAnswer={answers[currentQ]}
          onSelect={(answer) => setAnswers({...answers,
[currentQ]: answer})}
        />

        <div className="quiz-nav">
          {currentQ > 0 && (
            <button onClick={() => setCurrentQ(currentQ - 1)}>
              ← Previous
            </button>
          )}

          {currentQ < questions.length - 1 ? (
            <button
              onClick={() => setCurrentQ(currentQ + 1)}
              disabled={!answers[currentQ]}
            >
              Next →
            </button>
          ) : (
            <button
              onClick={handleSubmit}
              disabled={Object.keys(answers).length <
questions.length}
              className="submit-btn"
            >
              Submit Quiz
            </button>
          )}
        </div>
      </>
    ) : (
      <QuizResults
        questions={questions}
        userAnswers={answers}
        score={score}
      />
    )}
  </div>
);
}

```

PART 3: ADDICTION & ENGAGEMENT MECHANICS

How TikTok Hooks Users (And How We Apply It)

TikTok's Addiction Playbook:

1. **Instant Gratification** (video starts immediately)
2. **Variable Rewards** (never know what's next)
3. **Infinite Scroll** (no stopping point)
4. **Social Validation** (likes, comments)
5. **FOMO** (fear of missing out)
6. **Progress Illusion** (feels productive)

Our Ethical Adaptation:

TikTok Hook → DeepLearn Version → Why It Works

1. INSTANT GRATIFICATION

TikTok: Video plays in <1 second

DeepLearn: Video starts in <2 seconds

```
// Preload next video while current one plays
function VideoPlayer({ currentVideo, nextVideo }) {
  useEffect(() => {
    // Preload next video in background
    if (nextVideo) {
      const iframe = document.createElement('iframe');
      iframe.src =
`https://www.youtube.com/embed/${nextVideo.videoId}`;
      iframe.style.display = 'none';
      document.body.appendChild(iframe);
    }
  }, [nextVideo]);

  return (
    <div className="player-container">
      <iframe
        src={`https://www.youtube.com/embed/${currentVideo.videoId}?
autoplay=1`}
        allow="autoplay"
        className="video-player"
      />
    </div>
  );
}
```

Result: User opens app → Video playing in 2 seconds → Immediate dopamine

2. VARIABLE REWARDS

TikTok: Never know what video is next

DeepLearn: Never know what reward you'll get

```
const REWARDS = [
  { type: 'badge', rarity: 'common', message: '🎉 Video Complete!' },
  { type: 'badge', rarity: 'rare', message: '👏 3-Day Streak!' },
  { type: 'badge', rarity: 'epic', message: '🏆 Perfect Quiz Score!' },
  { type: 'unlock', rarity: 'rare', message: '🎁 Bonus Video Unlocked!' },
  { type: 'powerup', rarity: 'epic', message: '💖 Streak Freeze Earned!' },
  { type: 'leaderboard', rarity: 'legendary', message: '🏅 Top 10 This Week!' }
];
```

```

async function giveReward(userId, trigger) {
  // 70% chance of getting a reward
  if (Math.random() > 0.3) return;

  // Weighted random selection (rare rewards are... rare)
  const roll = Math.random();
  let rarity;

  if (roll < 0.60) rarity = 'common';    // 60%
  else if (roll < 0.85) rarity = 'rare';  // 25%
  else if (roll < 0.97) rarity = 'epic';  // 12%
  else rarity = 'legendary';             // 3%

  const possibleRewards = REWARDS.filter(r => r.rarity === rarity);
  const reward = possibleRewards[Math.floor(Math.random() *
possibleRewards.length)];

  // Save reward
  await supabase.from('user_rewards').insert({
    user_id: userId,
    type: reward.type,
    rarity: rarity,
    message: reward.message,
    earned_at: new Date()
  });

  // Show celebration animation
  showRewardAnimation(reward);

  // Play sound (dopamine trigger)
  playSound(rarity); // Different sounds for different rarities
}

function showRewardAnimation(reward) {
  // Confetti for epic/legendary
  if (reward.rarity === 'epic' || reward.rarity === 'legendary') {
    confetti({
      particleCount: 100,
      spread: 70,
      origin: { y: 0.6 }
    });
  }

  // Show popup with animation
  const popup = document.createElement('div');
  popup.className = `reward-popup ${reward.rarity}`;
  popup.innerHTML = `
    <div class="reward-icon">${reward.message.split(' ')[0]}</div>
    <div class="reward-text">${reward.message}</div>
  `;

  document.body.appendChild(popup);

  setTimeout(() => popup.classList.add('show'), 100);
  setTimeout(() => {
    popup.classList.remove('show');
    setTimeout(() => popup.remove(), 500);
  }, 3000);
}

```

Result: Users complete videos hoping for rare rewards → Dopamine lottery

3. STREAKS (FOMO + Loss Aversion)

TikTok: None (weakness)

DeepLearn: Duolingo-style streaks (proven addiction)

```

async function updateStreak(userId) {
  const { data: user } = await supabase
    .from('users')
    .select('*')

```

```

    .eq('id', userId)
    .single();

const today = new Date().toDateString();
const lastActive = new Date(user.last_active).toDateString();
const yesterday = new Date(Date.now() - 86400000).toDateString();

let newStreak = user.streak;
let streakStatus = 'maintained';

// If active today already, no change
if (today === lastActive) {
    return { streak: newStreak, status: 'already_counted' };
}

// If last active was yesterday, increment streak
if (lastActive === yesterday) {
    newStreak = user.streak + 1;
    streakStatus = 'increased';

    // Milestone celebrations
    if (newStreak === 7) {
        giveReward(userId, 'week_streak');
        showMessage('🔥 7-Day Streak! You\'re on fire!');
    } else if (newStreak === 30) {
        giveReward(userId, 'month_streak');
        showMessage('🦄 30-Day Streak! You\'re a legend!');
    } else if (newStreak === 100) {
        giveReward(userId, 'century_streak');
        showMessage('🏆 100-DAY STREAK! You\'re unstoppable!');
    }
}

// If last active was before yesterday
else {
    // Check if user has streak freezes
    if (user.streak_freezes > 0) {
        // Use freeze
        await supabase
            .from('users')
            .update({ streak_freezes: user.streak_freezes - 1 })
            .eq('id', userId);

        showMessage('💔 Streak Freeze used! Your streak is safe.');
```

streakStatus = 'frozen';

```

    } else {
        // Streak broken
        newStreak = 0;
        streakStatus = 'broken';

        showStreakBreakMessage(user.streak);
    }
}

// Update user
await supabase
    .from('users')
    .update({
        streak: newStreak,
        last_active: new Date()
    })
    .eq('id', userId);

return { streak: newStreak, status: streakStatus };
}

function showStreakBreakMessage(oldStreak) {
    // Make it HURT (loss aversion)
    const messages = [
        `💔 Your ${oldStreak}-day streak is broken...`,
        `😞 You were doing so well for ${oldStreak} days!`,
        `🕒 Missed a day... ${oldStreak}-day streak lost.`
    ];

    const message = messages[Math.floor(Math.random() *
messages.length)];

```

```

    showMessage(message, 'error');

    // Show what they could have had
    if (oldStreak >= 7) {
      showMessage(`But hey, you can start fresh! Get to 7 days again
to earn a Streak Freeze.`);
    }
  }

  // Give streak freezes as rewards
  async function grantWeeklyFreeze(userId) {
    const { data: user } = await supabase
      .from('users')
      .select('streak_freezes')
      .eq('id', userId)
      .single();

    // Max 2 freezes at a time
    if (user.streak_freezes < 2) {
      await supabase
        .from('users')
        .update({ streak_freezes: user.streak_freezes + 1 })
        .eq('id', userId);

      showMessage('💖 Streak Freeze earned! Use it if you miss a
day.');
```

Result: Users check in daily to maintain streak → Habit formation

4. SOCIAL PROOF & COMPETITION

TikTok: Likes, followers, views

DeepLearn: Leaderboards, study buddies

```

// Global Leaderboard
async function getLeaderboard(timeframe = 'week') {
  const startDate = timeframe === 'week'
    ? new Date(Date.now() - 7 * 86400000)
    : new Date(Date.now() - 30 * 86400000);

  const { data: topLearners } = await supabase
    .from('video_completions')
    .select('user_id, users(username, avatar)')
    .gte('completed_at', startDate.toISOString())
    .order('completed_at', { ascending: false });

  // Count completions per user
  const leaderboard = {};
  topLearners.forEach(completion => {
    const userId = completion.user_id;
    if (!leaderboard[userId]) {
      leaderboard[userId] = {
        username: completion.users.username,
        avatar: completion.users.avatar,
        count: 0
      };
    }
    leaderboard[userId].count++;
  });

  // Sort and return top 100
  return Object.values(leaderboard)
    .sort((a, b) => b.count - a.count)
    .slice(0, 100)
    .map((user, index) => ({
      rank: index + 1,
      ...user
    }));
}
```

```

// Accountability Partner System
async function createStudyBuddy(userId1, userId2) {
  await supabase.from('study_buddies').insert({
    user1_id: userId1,
    user2_id: userId2,
    created_at: new Date()
  });

  // Set up mutual notifications
  setupBuddyNotifications(userId1, userId2);
}

async function setupBuddyNotifications(user1, user2) {
  // If one user completes a video, notify the other
  supabase
    .channel(`buddy-${user1}-${user2}`)
    .on('postgres_changes', {
      event: 'INSERT',
      schema: 'public',
      table: 'video_completions',
      filter: `user_id=eq.${user1}`
    }, (payload) => {
      sendNotification(user2, {
        title: 'Your study buddy is crushing it!',
        body: `They just completed another video. Don't fall
behind!`,
        action: 'OPEN_APP'
      });
    })
    .subscribe();
}

// Show position to create urgency
function LeaderboardWidget({ currentUser }) {
  const [position, setPosition] = useState(null);
  const [nextUser, setNextUser] = useState(null);

  useEffect(() => {
    async function fetchPosition() {
      const board = await getLeaderboard('week');
      const myPos = board.findIndex(u => u.username ===
currentUser.username);

      setPosition(myPos + 1);

      if (myPos > 0) {
        setNextUser(board[myPos - 1]);
      }
    }

    fetchPosition();
  }, []);

  return (
    <div className="leaderboard-widget">
      <h3>Your Rank This Week</h3>
      <div className="rank-display">#{position || '-'}</div>

      {nextUser && (
        <div className="catch-up">
          <p>Complete {nextUser.count - (position - 1)} more videos
to reach #{position - 1}</p>
          <button>🏃 Let's Go!</button>
        </div>
      )}
    </div>
  );
}

```

Result: Competition + social pressure → Increased engagement

5. PROGRESS VISUALIZATION

TikTok: Watch time counter

DeepLearn: Skill tree, completion rings

```
// Skill Tree Visualization
function SkillTree({ userId, topic }) {
  const [nodes, setNodes] = useState([]);

  useEffect(() => {
    async function loadProgress() {
      const { data: completions } = await supabase
        .from('video_completions')
        .select('video_id, videos(title, day_number)')
        .eq('user_id', userId)
        .eq('videos.topic', topic);

      // Create tree nodes
      const treeNodes = Array.from({ length: 30 }, (_, i) => ({
        day: i + 1,
        completed: completions.some(c => c.videos.day_number === i +
1),
        locked: i + 1 > (completions.length + 1)
      }));

      setNodes(treeNodes);
    }

    loadProgress();
  }, [userId, topic]);

  return (
    <svg className="skill-tree" viewBox="0 0 800 1200">
      {/* Draw connections */}
      {nodes.map((node, i) => {
        if (i === 0) return null;

        const x1 = 400;
        const y1 = i * 40;
        const x2 = 400;
        const y2 = (i + 1) * 40;

        return (
          <line
            key={`line-${i}`}
            x1={x1}
            y1={y1}
            x2={x2}
            y2={y2}
            stroke={node.completed ? '#10b981' : '#ddd'}
            strokeWidth="3"
          />
        );
      })}

      {/* Draw nodes */}
      {nodes.map((node, i) => {
        const x = 400;
        const y = i * 40;

        let fill = '#ddd'; // Locked
        if (node.completed) fill = '#10b981'; // Completed (green)
        else if (!node.locked) fill = '#3b82f6'; // Available
        (blue)

        return (
          <g key={`node-${i}`}>
            <circle cx={x} cy={y} r="15" fill={fill} />
            <text x={x} y={y + 5} textAnchor="middle" fill="white"
fontSize="12">
              {node.day}
            </text>
          </g>
        );
      })}
    </svg>
  );
}
```

```

    })}
  </svg>
  );
}

// Completion Ring (like Apple Watch)
function CompletionRing({ completed, total }) {
  const percentage = (completed / total) * 100;
  const circumference = 2 * Math.PI * 45; // radius = 45
  const offset = circumference - (percentage / 100) * circumference;

  return (
    <svg className="completion-ring" width="120" height="120">
      { /* Background circle */ }
      <circle
        cx="60"
        cy="60"
        r="45"
        fill="none"
        stroke="#e5e7eb"
        strokeWidth="10"
      />

      { /* Progress circle */ }
      <circle
        cx="60"
        cy="60"
        r="45"
        fill="none"
        stroke="#10b981"
        strokeWidth="10"
        strokeDasharray={circumference}
        strokeDashoffset={offset}
        strokeLinecap="round"
        transform="rotate(-90 60 60)"
      />

      { /* Center text */ }
      <text x="60" y="65" textAnchor="middle" fontSize="24"
fontWeight="bold">
        {completed}/{total}
      </text>
    </svg>
  );
}

```

Result: Visual progress → Completion motivation

6. PUSH NOTIFICATIONS (Perfectly Timed)

TikTok: Random “Someone liked your video”

DeepLearn: Smart, context-aware notifications

```

// Notification Timing Strategy
const NOTIFICATION_SCHEDULE = {
  morning: {
    time: '09:00',
    messages: [
      '☀️ Good morning! Ready to learn something new?',
      '☕ Coffee ready? So is your next video!',
      '🕒 Start your day with a quick 15-minute lesson'
    ]
  },
  lunchtime: {
    time: '13:00',
    messages: [
      '🌞 Lunch break? Perfect time for a learning snack!',
      '🕒 Halfway through the day. Halfway through your video?'
    ]
  },
}

```



```

evening: {
  time: '20:00',
  messages: [
    '🌙 Evening wind-down with some knowledge?',
    '📺 Don\'t let your streak die! 4 hours left.',
    '🔔 Your {streak}-day streak is waiting...'
  ]
},

urgent: {
  time: '23:00',
  condition: 'streak_at_risk',
  messages: [
    '⚠️ 1 HOUR LEFT! Don\'t break your {streak}-day streak!',
    '📺 Your streak is about to break! Quick 15-min video?'
  ]
}
};

async function sendSmartNotification(userId) {
  const { data: user } = await supabase
    .from('users')
    .select('*')
    .eq('id', userId)
    .single();

  const now = new Date();
  const today = now.toDateString();
  const lastActive = new Date(user.last_active).toDateString();

  // Don't notify if already active today
  if (today === lastActive) return;

  // Choose notification based on time
  const hour = now.getHours();
  let notif;

  if (hour >= 8 && hour < 12) {
    notif = NOTIFICATION_SCHEDULE.morning;
  } else if (hour >= 12 && hour < 17) {
    notif = NOTIFICATION_SCHEDULE.lunchtime;
  } else if (hour >= 19 && hour < 22) {
    notif = NOTIFICATION_SCHEDULE.evening;
  } else if (hour >= 23 || hour < 1) {
    // URGENT: Streak about to break
    notif = NOTIFICATION_SCHEDULE.urgent;
  } else {
    return; // Don't spam at weird hours
  }

  // Personalize message
  const message = notif.messages[Math.floor(Math.random() *
notif.messages.length)]
    .replace('{streak}', user.streak);

  // Send notification
  await sendPushNotification(user.fcm_token, {
    title: 'DeepLearn',
    body: message,
    data: { action: 'OPEN_APP' }
  });
}

// Schedule daily check
cron.schedule('0 9,13,20,23 * * *', async () => {
  // Get all users who haven't completed today
  const { data: inactiveUsers } = await supabase
    .from('users')
    .select('id')
    .lt('last_active', new Date().toDateString());

  // Send notifications
  for (const user of inactiveUsers) {
    await sendSmartNotification(user.id);
  }
});

```

```
}  
});
```

Result: Timely reminders → Daily engagement

7. ONBOARDING THAT HOOKS

TikTok: Skip tutorial, dive in

DeepLearn: Interactive 60-second onboarding

```
function OnboardingFlow() {  
  const [step, setStep] = useState(0);  
  const [userGoal, setUserGoal] = useState('');  
  const [preferredTime, setPreferredTime] = useState('');  
  
  const steps = [  
    {  
      title: 'Welcome to DeepLearn! 🎓',  
      description: 'Most people quit online courses. You won\'t.  
Here\'s why:',  
      component: <WelcomeAnimation />  
    },  
    {  
      title: 'What do you want to master?',  
      component: (  
        <input  
          placeholder="e.g., React, Python, Machine Learning"  
          value={userGoal}  
          onChange={(e) => setUserGoal(e.target.value)}  
          className="goal-input"  
        />  
      )  
    },  
    {  
      title: 'When do you learn best?',  
      component: (  
        <div className="time-picker">  
          <button onClick={() => setPreferredTime('morning')}>  
            🌞 Morning (8-12)  
          </button>  
          <button onClick={() => setPreferredTime('afternoon')}>  
            * Afternoon (12-17)  
          </button>  
          <button onClick={() => setPreferredTime('evening')}>  
            🌙 Evening (18-23)  
          </button>  
        </div>  
      )  
    },  
    {  
      title: 'Perfect! Here\'s your first video.',  
      description: 'Watch it, pass the quiz, write a summary.  
That\'s it!',  
      component: <FirstVideoPreview goal={userGoal} />  
    }  
  ];  
  
  return (  
    <div className="onboarding">  
      {steps[step].component}  
  
      <button  
        onClick={() => {  
          if (step === steps.length - 1) {  
            completeOnboarding(userGoal, preferredTime);  
          } else {  
            setStep(step + 1);  
          }  
        }}  
        className="next-btn"  
      >  
        {step === steps.length - 1 ? 'Start Learning! 🚀' : 'Next'  

```

```

→'}
      </button>
    </div>
  );
}

async function completeOnboarding(goal, time) {
  // Create personalized learning path
  const path = await generateLearningPath(goal, 'beginner', 30);

  // Save user preferences
  await supabase.from('users').update({
    goal: goal,
    preferred_time: time,
    onboarding_complete: true
  }).eq('id', currentUser.id);

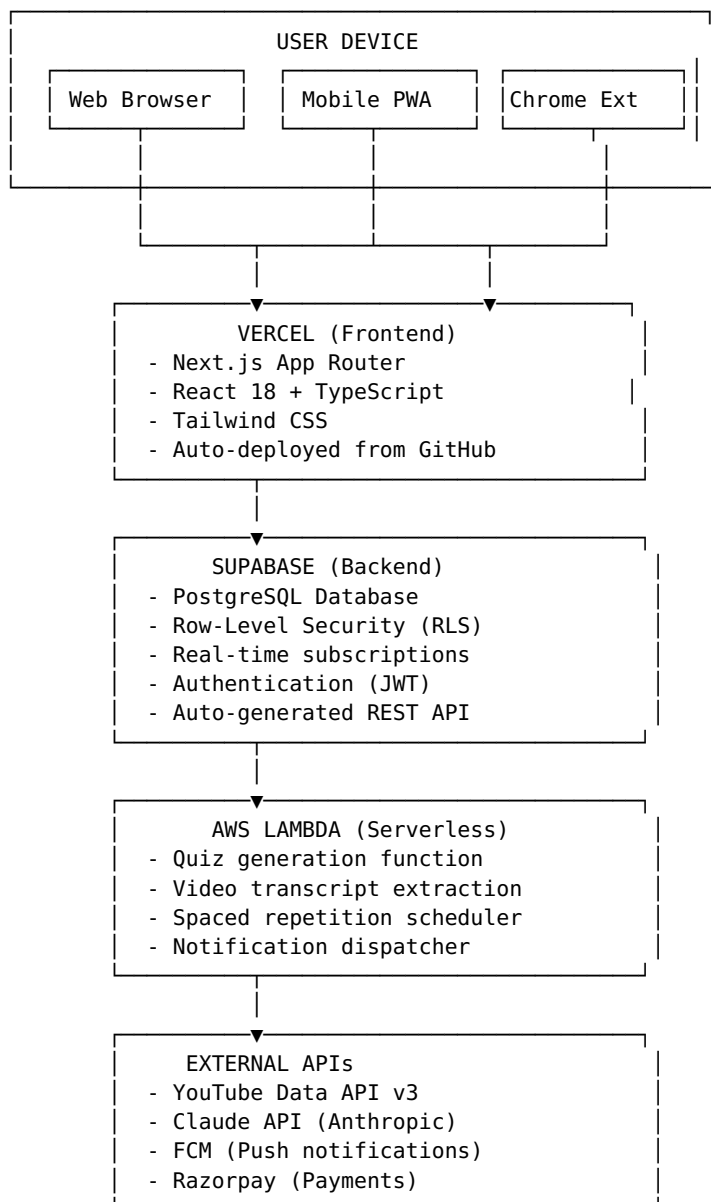
  // Start first video IMMEDIATELY
  router.push(`/learn/${path.path[0].video.videoId}`);
}

```

Result: Fast onboarding → Immediate value → Hook set

PART 4: COMPLETE TECHNICAL STACK

Architecture Diagram



Database Schema

```
-- Users Table
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email VARCHAR(255) UNIQUE NOT NULL,
  username VARCHAR(50) UNIQUE NOT NULL,
  avatar_url TEXT,

  -- Learning preferences
  goal TEXT,
  preferred_time VARCHAR(20),
  user_level VARCHAR(20) DEFAULT 'beginner',

  -- Engagement metrics
  streak INT DEFAULT 0,
  longest_streak INT DEFAULT 0,
  streak_freezes INT DEFAULT 0,
  last_active TIMESTAMP,

  -- Subscription
  subscription_tier VARCHAR(20) DEFAULT 'free',
  subscription_ends_at TIMESTAMP,

  -- Metadata
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Videos Table
CREATE TABLE videos (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  video_id VARCHAR(50) UNIQUE NOT NULL, -- YouTube video ID
  platform VARCHAR(20) DEFAULT 'youtube',

  -- Video metadata
  title TEXT NOT NULL,
  channel TEXT,
  duration INT, -- in seconds
  thumbnail_url TEXT,

  -- Quality metrics (from our data)
  avg_quiz_score DECIMAL(5,2),
  completion_count INT DEFAULT 0,
  quality_score DECIMAL(10,2),

  -- Topic classification
  topic VARCHAR(100),
  difficulty VARCHAR(20),

  created_at TIMESTAMP DEFAULT NOW()
);

-- Learning Paths Table
CREATE TABLE learning_paths (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),

  -- Path details
  topic TEXT NOT NULL,
  total_days INT,
  difficulty VARCHAR(20),

  -- Status
  current_day INT DEFAULT 1,
  completed BOOLEAN DEFAULT FALSE,

  created_at TIMESTAMP DEFAULT NOW(),
  completed_at TIMESTAMP
);

-- Path Videos Table (junction)
```

```

CREATE TABLE path_videos (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  path_id UUID REFERENCES learning_paths(id),
  video_id UUID REFERENCES videos(id),

  day_number INT NOT NULL,
  learning_objectives TEXT[],

  UNIQUE(path_id, day_number)
);

-- Video Completions Table
CREATE TABLE video_completions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  video_id UUID REFERENCES videos(id),
  path_id UUID REFERENCES learning_paths(id),

  -- Completion data
  watch_time INT, -- seconds watched
  quiz_score INT, -- 0-100
  summary TEXT,

  -- Timestamps
  started_at TIMESTAMP,
  completed_at TIMESTAMP DEFAULT NOW(),

  UNIQUE(user_id, video_id)
);

-- Quizzes Table
CREATE TABLE quizzes (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  video_id UUID REFERENCES videos(id),

  -- Quiz content
  questions JSONB NOT NULL, -- Array of question objects
  difficulty VARCHAR(20),

  -- Metadata
  generated_by VARCHAR(50) DEFAULT 'claude-sonnet-4',
  generated_at TIMESTAMP DEFAULT NOW(),

  UNIQUE(video_id, difficulty)
);

-- Quiz Attempts Table
CREATE TABLE quiz_attempts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  quiz_id UUID REFERENCES quizzes(id),
  video_id UUID REFERENCES videos(id),

  -- Attempt data
  attempt_number INT,
  answers JSONB, -- {questionId: selectedAnswer}
  score INT,
  passed BOOLEAN,
  time_taken INT, -- seconds

  completed_at TIMESTAMP DEFAULT NOW()
);

-- Spaced Repetition Reviews Table
CREATE TABLE spaced_reviews (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  video_id UUID REFERENCES videos(id),

  -- Review schedule
  review_number INT, -- 1, 2, 3 (for 1-day, 3-day, 7-day)
  scheduled_for TIMESTAMP,
  completed BOOLEAN DEFAULT FALSE,
  completed_at TIMESTAMP,

```

```

-- Review performance
recall_score INT, -- 0-100, how well they remembered

created_at TIMESTAMP DEFAULT NOW()
);

-- User Rewards Table
CREATE TABLE user_rewards (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),

  -- Reward details
  type VARCHAR(50), -- badge, powerup, unlock
  rarity VARCHAR(20), -- common, rare, epic, legendary
  message TEXT,
  icon TEXT,

  earned_at TIMESTAMP DEFAULT NOW()
);

-- Study Buddies Table
CREATE TABLE study_buddies (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user1_id UUID REFERENCES users(id),
  user2_id UUID REFERENCES users(id),

  -- Status
  status VARCHAR(20) DEFAULT 'active', -- active, inactive

  created_at TIMESTAMP DEFAULT NOW(),

  UNIQUE(user1_id, user2_id)
);

-- Notifications Table
CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),

  -- Notification content
  title TEXT,
  body TEXT,
  type VARCHAR(50),
  data JSONB,

  -- Status
  sent BOOLEAN DEFAULT FALSE,
  read BOOLEAN DEFAULT FALSE,

  scheduled_for TIMESTAMP,
  sent_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Create indexes for performance
CREATE INDEX idx_users_streak ON users(streak DESC);
CREATE INDEX idx_videos_quality ON videos(quality_score DESC);
CREATE INDEX idx_completions_user ON video_completions(user_id,
completed_at DESC);
CREATE INDEX idx_reviews_scheduled ON spaced_reviews(user_id,
scheduled_for) WHERE completed = FALSE;

```

API Endpoints

```

// Authentication
POST /api/auth/signup
POST /api/auth/login
POST /api/auth/logout
GET /api/auth/me

// Learning Paths
POST /api/paths/generate // Generate new learning path

```

```
GET    /api/paths/:pathId
GET    /api/paths/user/:userId    // Get user's active paths
PUT    /api/paths/:pathId/advance // Move to next day

// Videos
GET    /api/videos/search        // Search YouTube
GET    /api/videos/:videoId
POST   /api/videos/add          // Add video to user's queue
GET    /api/videos/recommend  // Get recommendations

// Quizzes
GET    /api/quizzes/:videoId    // Get or generate quiz
POST   /api/quizzes/:quizId/attempt // Submit quiz attempt
GET    /api/quizzes/attempts/:userId // Get attempt history

// Progress
GET    /api/progress/:userId
POST   /api/progress/complete    // Mark video complete
GET    /api/progress/streak/:userId

// Spaced Repetition
GET    /api/reviews/due/:userId // Get reviews due today
POST   /api/reviews/:reviewId/complete

// Social
GET    /api/leaderboard        // Global leaderboard
POST   /api/buddies/invite     // Invite study buddy
GET    /api/buddies/:userId    // Get user's buddies

// Payments
POST   /api/payments/checkout    // Create Razorpay order
POST   /api/payments/verify     // Verify payment
GET    /api/payments/subscription/:userId
```

Tech Stack Summary

Layer	Technology	Cost	Why
Frontend	Next.js 14 + React	FREE	Server components, fast
Styling	Tailwind CSS	FREE	Rapid UI development
Hosting	Vercel	FREE	Auto-deploy, edge network
Database	Supabase PostgreSQL	FREE*	Real-time, auth built-in
Serverless	AWS Lambda	\$20-500	Pay per use, scales auto
AI	Claude API (Anthropic)	\$100-1K	Best quiz generation
Video API	YouTube Data API v3	FREE	10K requests/day free
Payments	Razorpay	2% fee	Indian market leader
Push Notifs	Firebase Cloud Messaging	FREE	Reliable delivery
Analytics	Plausible	\$9/mo	Privacy-first analytics

Total Monthly Cost (1K users): \$150-200
Total Monthly Cost (100K users): \$1,200-1,500

PART 5: 12-WEEK

IMPLEMENTATION ROADMAP

Week 1-2: Foundation

Goals: - Set up development environment - Build core video player - Implement YouTube API integration

Tasks:

- [x] Create Next.js project with TypeScript
- [x] Set up Supabase project
- [x] Configure Tailwind CSS
- [x] Build video player component
- [x] Integrate YouTube Iframe API
- [x] Test video embedding
- [x] Create basic routing structure

Deliverable: Video player that can play YouTube videos

Week 3-4: AI Quiz Generation

Goals: - Integrate Claude API - Build quiz generation pipeline - Create quiz UI

Tasks:

- [x] Set up Claude API credentials
- [x] Build transcript extraction function
- [x] Create quiz generation prompt
- [x] Test quiz quality
- [x] Build quiz UI components
- [x] Implement quiz evaluation logic
- [x] Add results display

Deliverable: Working AI-generated quizzes

Week 5-6: Core Learning Loop

Goals: - Sequential locking - Summary requirement - Streak tracking

Tasks:

- [x] Implement sequential unlock logic
- [x] Build summary input component
- [x] Create completion validation
- [x] Build streak counter
- [x] Add daily limit enforcement
- [x] Test full learning loop

Deliverable: Complete “watch → quiz → summary → unlock” flow

Week 7-8: Engagement Features

Goals: - Spaced repetition - Rewards system - Progress visualization

Tasks:

- [x] Build spaced repetition scheduler
- [x] Create review notification system
- [x] Implement rewards/badges
- [x] Build progress dashboard
- [x] Add skill tree visualization
- [x] Create completion rings

Deliverable: Gamified learning experience

Week 9-10: Social & Monetization

Goals: - Leaderboards - Payment integration - Freemium gates

Tasks:

- [x] Build leaderboard system
- [x] Integrate Razorpay
- [x] Create subscription tiers
- [x] Implement paywall logic
- [x] Build checkout flow
- [x] Add study buddy feature

Deliverable: Revenue-generating platform

Week 11-12: Polish & Launch

Goals: - Bug fixes - Performance optimization - Marketing prep

Tasks:

- [x] Comprehensive testing
- [x] Mobile responsiveness
- [x] Performance optimization
- [x] SEO setup
- [x] Create landing page
- [x] Prepare launch materials
- [x] Product Hunt submission

Deliverable: Production-ready app

CONCLUSION

What Makes DeepLearn Different

Other Platforms: - Maximize watch time → Profit from ads - Endless content → Overwhelm users - No accountability → High dropout

DeepLearn: - Maximize retention → Profit from results - Limited content → Focus users - Forced accountability → High completion

Success Metrics (6 Months)

- **Users:** 10,000+
- **Day 7 Retention:** 40%+
- **Conversion Rate:** 5-8%
- **Monthly Revenue:** ₹50K+
- **Avg. Completion Rate:** 60%+ (vs 5% industry)

The Vision

We're not just building an app.

We're creating a rebellion against the attention economy.

We're proving that constraint breeds mastery.

We're helping 95% of learners who start courses actually finish them.

This is how we make learning as addictive as TikTok—but instead of wasting time, users gain knowledge.

Let's build this. 🦋

